SmartPetHome

Device Integration Guide

Sprint 2 — For developers and AI agents integrating ESP32 hardware

SmartPetHome — Device Integration Guide

Version: Sprint 2

Audience: Developers and AI agents implementing new ESP32 devices or integrating hardware with the SmartPetHome backend.

Table of Contents

---

1. System Overview

Telemetry flow (device → app)

```
ESP32 sensor
  → reads hardware (ADC, GPIO, I2C, etc.)
  → builds JSON payload
  → publishes to MQTT topic via WiFi
    ↓
EMQX Public Broker (broker.emqx.io:1883)
    ↓
Flask backend (mqtt_client.py subscribes on start)
  → parses topic → extracts serial_number and message type
  → calls telemetry_handlers.dispatch_telemetry()
  → handler inserts row into event table (e.g. feeding_events)
  → checks alert thresholds → inserts alerts table row if triggered
  → sends Telegram notification if chat_id is configured
  → updates devices.last_seen_at and devices.status = "online"
    ↓
Supabase PostgreSQL
```

```
    ↓
Web app (JavaScript polls /api/dashboard/summary and /api/telemetry every 5 s)
```

Command flow (app → device)

```
User clicks command button in web app
  → POST /api/devices/{device_id}/command
  → Flask builds command JSON with a UUID command_id
  → publishes to smartpethome/devices/{serial_number}/command
    ↓
EMQX Public Broker
    ↓
ESP32 receives via mqttClient.loop()
  → parses command JSON
  → executes action (servo, valve, etc.)
  → publishes ACK JSON to smartpethome/devices/{serial_number}/ack
    ↓
Flask backend receives ACK
  → stores in ack_store dict keyed by command_id
  → for dispense_food ACK: inserts manual_dispense feeding_event
    ↓
Web app polls /api/devices/{device_id}/ack/{command_id}
  → shows ■ or ■ result to user
```

---

2. MQTT Broker Configuration

The project uses the **EMQX Public Broker** (no authentication, no TLS). This is a shared public broker suitable for university prototypes. Do not send sensitive data.

```
MQTT_HOST=broker.emqx.io
MQTT_PORT=1883
MQTT_USERNAME=
MQTT_PASSWORD=
MQTT_TLS=false
MQTT_CLIENT_ID=smartpethome-backend
```

> ***Important:*** *The Flask backend appends a random 6-character suffix to* `MQTT_CLIENT_ID` *at runtime (e.g.* `smartpethome-backend-k3xz9q`*). This prevents EMQX from banning duplicate client IDs when the app is restarted. Each ESP32 must use its own unique static client ID (e.g.* `feeder-001-esp32`*).*

ESP32 MQTT settings

```
#define MQTT_HOST  "broker.emqx.io"
#define MQTT_PORT  1883
// No username, no password, no TLS
mqttClient.setBufferSize(512);  // REQUIRED — default 256 is too small
```

> ***Critical:*** *Call* `mqttClient.setBufferSize(512)` *before* `mqttClient.connect()`*. The default 256-byte buffer silently drops payloads larger than ~256 bytes. The feeder telemetry payload is ~286 bytes.*

---

3. Global Topic Structure

All topics follow the same pattern regardless of device type:

Direction	Topic	Purpose
ESP32 → Backend	`smartpethome/devices/{serial_number}/telemetry`	Sensor readings, every 1 s recommended
ESP32 → Backend	`smartpethome/devices/{serial_number}/status`	Heartbeat, every 10 s recommended
Backend → ESP32	`smartpethome/devices/{serial_number}/command`	Commands from the web app
ESP32 → Backend	`smartpethome/devices/{serial_number}/ack`	Command acknowledgement

`{serial_number}` **must exactly match the serial registered in the SmartPetHome app.** The backend looks up the device by serial number in Supabase. If there is no match, the telemetry is ignored and a warning is logged.

Example for device with serial `ENV-001`

```
smartpethome/devices/ENV-001/telemetry
smartpethome/devices/ENV-001/status
smartpethome/devices/ENV-001/command
smartpethome/devices/ENV-001/ack
```

---

4. Status Payload (all devices)

Publish to `smartpethome/devices/{serial_number}/status` every 10 s.

```
{
  "serial_number": "FEEDER-001",
  "status": "online",
  "uptime_seconds": 3721,
  "wifi_rssi": -55
}
```

Field	Type	Required	Notes
`serial_number`	string	Yes	Must match app registration
`status`	string	Yes	One of: `online`, `offline`, `error`, `maintenance`
`uptime_seconds`	number	No	Seconds since last boot
`wifi_rssi`	number	No	WiFi signal strength in dBm

The backend updates `devices.last_seen_at` and `devices.status` on every status message.

---

5. ACK Payload (all devices)

Publish to `smartpethome/devices/{serial_number}/ack` after executing a command.

```
{
  "command_id": "550e8400-e29b-41d4-a716-446655440000",
```

```
  "status": "ok",
  "message": "Food dispensed successfully"
}
```

Field	Type	Required	Notes
`command_id`	string	Yes	Must match the `command_id` from the received command
`status`	string	Yes	`"ok"` or `"success"` = success; `"error"` = failure
`message`	string	No	Human-readable result shown in the app

The backend accepts both `"ok"` and `"success"` as successful ACK statuses.

---

6. Command Payload (all devices)

Received on `smartpethome/devices/{serial_number}/command`.

```
{
  "command": "dispense_food",
  "command_id": "550e8400-e29b-41d4-a716-446655440000",
  "params": {
    "grams": 50
  }
}
```

Field	Type	Required	Notes
`command`	string	Yes	Command name — see per-device tables below
`command_id`	string	Yes	UUID — echo this back in the ACK
`params`	object	Yes	May be empty `{}` for parameterless commands

---

7. Device Registration Steps

- Log in to SmartPetHome.
- Go to **Devices** → **Register a Device**.
- Enter a **unique serial number** (e.g. `FEEDER-001`, `ENV-001`).
- Enter a device name (e.g. "Kitchen Feeder").
- Select the correct **device type** from the dropdown.
- If device type is **Automatic Feeder**, assign it to a pet (required).
- Click **Register Device**.
- Copy the exact serial number into your ESP32 sketch.
- Flash the ESP32 and connect to WiFi.
- The device will appear **online** in the dashboard within 10 s of the first status message.

> *If the serial number is already registered (by any user), registration will fail with a friendly error. Use a globally unique serial — prefixing with your project name avoids collisions on the shared broker.*

---

8. Device Type Reference

8.1 `automatic_feeder`

Purpose: Monitors food bowl weight and dispenses food on command.

Physical prototype: ESP32 + potentiometer (weight sensor) + servo (gate) + red LED (low food) + green LED (normal).

Reference sketch: `esp32/feeder_001/feeder_001.ino`

Telemetry topic

```
smartpethome/devices/{serial_number}/telemetry
```

Telemetry payload

```
{
  "serial_number": "FEEDER-001",
  "device_type": "automatic_feeder",
  "data": {
    "bowl_weight_grams":    1850,
    "food_remaining_grams": 1850,
    "dispensed_grams":      0,
    "consumed_grams":       0,
    "leftover_grams":       0,
    "status_color":         "green",
    "low_food":             false,
    "pet_underfed":         false
  }
}
```

Field	Type	Notes
`bowl_weight_grams`	number	Raw ADC reading (0–4095 for ESP32 12-bit ADC)
`food_remaining_grams`	number	Same as bowl_weight_grams in prototype
`dispensed_grams`	number	Grams dispensed this cycle (0 in prototype)
`consumed_grams`	number	Grams consumed since last fill (0 in prototype)
`leftover_grams`	number	Grams remaining after consumption (0 in prototype)
`status_color`	string	`"red"` if low, `"green"` if normal
`low_food`	boolean	`true` if reading < `umbralPeso` (1800)
`pet_underfed`	boolean	`true` if pet may not have eaten enough

Alert thresholds (backend)

Reading range	Alert triggered
> 3000	No alert
1800 – 3000	`low_food` WARNING alert (once per event)
< 1800	`pet_underfed` CRITICAL alert (once per event)

Alerts are deduplicated: a second alert of the same type is not created while the first remains unresolved.

Backend event table

`feeding_events` — rows inserted only on **state transitions** (ok→low, low→ok), not every second. Manual dispenses are recorded when the ACK arrives.

Commands

Command	Params	Description
`dispense_food`	`{"grams": 50}`	Opens gate servo for 2 s, then closes
`set_feeding_mode`	`{"mode": "complete_bowl"}`	Set mode: `complete_bowl` or `redistribute_daily_diet`
`sync_schedules`	`{}`	Push feeding schedule to device
`calibrate_scale`	`{}`	Trigger scale calibration
`tare_scale`	`{}`	Zero the scale
`get_status`	`{}`	Trigger immediate status + telemetry publish

Command example (backend → ESP32)

```
{
  "command": "dispense_food",
  "command_id": "550e8400-e29b-41d4-a716-446655440000",
  "params": { "grams": 50 }
}
```

ACK example (ESP32 → backend)

```
{
  "command_id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "ok",
  "message": "Food dispensed"
}
```

8.2 `water_dispenser`

Purpose: Monitors water level and controls a refill valve.

Telemetry payload

```
{
  "serial_number": "WATER-001",
```

```
  "device_type": "water_dispenser",
  "data": {
    "water_level":        75,
    "water_level_before": 80,
    "water_level_after":  75,
    "refill_triggered":   false,
    "supply_failure":     false,
    "valve_state":        "closed",
    "low_water":          false
  }
}
```

Field	Type	Notes
`water_level`	number	Current level (0–100%)
`water_level_before`	number	Level before last refill cycle
`water_level_after`	number	Level after last refill cycle
`refill_triggered`	boolean	`true` if auto-refill ran this cycle
`supply_failure`	boolean	`true` if water supply failed
`valve_state`	string	`"open"` or `"closed"`
`low_water`	boolean	`true` triggers a WARNING alert

Alert conditions

- `low_water: true` → `low_water` WARNING alert
- `supply_failure: true` → `failed_water_refill` CRITICAL alert

Backend event table: `water_events`

Commands

Command	Params	Description
`refill_now`	`{}`	Trigger immediate refill
`open_valve`	`{}`	Open the water valve
`close_valve`	`{}`	Close the water valve
`set_auto_refill`	`{"enabled": "true"}`	Enable or disable auto-refill
`get_status`	`{}`	Trigger status publish

8.3 `motion_monitoring_network`

Purpose: Detects pet motion and tracks inactivity periods across one or more sensors.

Telemetry payload

```
{
  "serial_number": "MOTION-001",
  "device_type": "motion_monitoring_network",
  "data": {
```

```
    "motion_detected":    true,
    "sensor_code":        "S1",
    "detected_at":        "2025-06-04T18:30:00Z",
    "inactivity_minutes": 0,
    "sensor_status":      "active"
  }
}
```

Field	Type	Notes
`motion_detected`	boolean	`true` when motion is detected
`sensor_code`	string	Identifies which sensor triggered (optional)
`detected_at`	ISO 8601 string	Timestamp of detection; defaults to server time if omitted
`inactivity_minutes`	number	Minutes since last motion detection
`sensor_status`	string	`"active"`, `"inactive"`, etc.

Alert conditions

- `inactivity_minutes >= threshold` (configured per-device in `motion_network_configurations`, default 60 min) → `inactivity` WARNING alert

Backend event table: `motion_events`

Commands

Command	Params	Description
`set_inactivity_limit`	`{"minutes": 45}`	Set inactivity alert threshold
`enable_monitoring`	`{}`	Start motion detection
`disable_monitoring`	`{}`	Stop motion detection
`get_status`	`{}`	Trigger status publish

8.4 `audio_communication`

Purpose: Plays audio messages or alerts through a speaker to communicate with the pet.

Telemetry payload

```
{
  "serial_number": "AUDIO-001",
  "device_type": "audio_communication",
  "data": {
    "status":             "played",
    "audio_file":         "morning_greeting.mp3",
    "volume_level":       75,
    "playback_started":   "2025-06-04T08:00:00Z",
    "playback_finished":  "2025-06-04T08:00:05Z",
    "error_message":      null
  }
}
```

Field	Type	Notes
`status`	string	One of: `sent`, `delivered`, `played`, `failed`
`audio_file`	string	Filename or message ID
`volume_level`	number	0–100
`playback_started`	ISO 8601 string	When audio started
`playback_finished`	ISO 8601 string	When audio ended (stored as `played_at`)
`error_message`	string or null	Error description if `status = "failed"`

Alert conditions

- `status == "failed"` or `error_message` is non-null → `audio_playback_failed` WARNING alert

Backend event table: `audio_events`

Commands

Command	Params	Description
`play_audio`	`{"audio_file": "greeting.mp3"}`	Play a specific audio file
`set_volume`	`{"level": 80}`	Set speaker volume (0–100)
`stop_audio`	`{}`	Stop current playback
`get_status`	`{}`	Trigger status publish

8.5 `environmental_monitor`

Purpose: Monitors temperature and environmental conditions; controls an actuator (heater, fan, AC) to keep the environment in range.

> ***Teammate answer:*** *For a device with serial* `ENV-001`*, the backend expects these exact topics:*
> ```
> *smartpethome/devices/ENV-001/telemetry*
> *smartpethome/devices/ENV-001/status*
> *smartpethome/devices/ENV-001/command*
> *smartpethome/devices/ENV-001/ack*
> ```

Telemetry payload

```
{
  "serial_number": "ENV-001",
  "device_type": "environmental_monitor",
  "data": {
    "temperature":        24.5,
    "humidity":           60,
    "status":             "normal",
```

```
    "actuator_triggered": false,
    "actuator_state":     "off",
    "min_temperature":    18.0,
    "max_temperature":    28.0
  }
}
```

Field	Type	Notes
`temperature`	number	Current temperature in °C — **required for alert logic**
`humidity`	number	Relative humidity 0–100% (optional)
`status`	string	One of: `normal`, `too_low`, `too_high`
`actuator_triggered`	boolean	`true` if auto-control fired the actuator this cycle
`actuator_state`	string	`"on"` or `"off"`
`min_temperature`	number	Lower safe bound (informational)
`max_temperature`	number	Upper safe bound (informational)

Alert conditions

- `status == "too_low"` or `status == "too_high"` → `temperature_out_of_range` CRITICAL alert

Backend event table: `environmental_events`

Commands

Command	Params	Description
`set_temperature_range`	`{"min_temperature": 18, "max_temperature": 28}`	Set safe temperature range
`turn_actuator_on`	`{}`	Manually turn the actuator on
`turn_actuator_off`	`{}`	Manually turn the actuator off
`set_auto_control`	`{"enabled": "true"}`	Enable or disable automatic temperature control
`get_status`	`{}`	Trigger status publish

Command example

```
{
  "command": "set_temperature_range",
  "command_id": "abc12345-...",
  "params": {
    "min_temperature": 18,
    "max_temperature": 28
  }
}
```

8.6 `automatic_access_door`

Purpose: Controls a pet door — opens or closes on command or on schedule.

Telemetry payload

```
{
  "serial_number": "DOOR-001",
  "device_type": "automatic_access_door",
  "data": {
    "action":        "open",
    "source":        "manual",
    "success":       true,
    "door_state":    "open",
    "error_message": null
  }
}
```

Field	Type	Notes
`action`	string	`"open"` or `"close"`
`source`	string	`"manual"`, `"scheduled"`, or `"sensor"`
`success`	boolean	Whether the action succeeded
`door_state`	string	Current state: `"open"` or `"closed"`
`error_message`	string or null	Error detail if `success = false`

Alert conditions

- `success == false` → `door_failed_to_open` or `door_failed_to_close` CRITICAL alert (based on `action`)

Backend event table: `access_door_events`

Commands

Command	Params	Description
`open_door`	`{}`	Open the door
`close_door`	`{}`	Close the door
`enable_manual_control`	`{}`	Allow manual physical override
`disable_manual_control`	`{}`	Lock out manual override
`get_status`	`{}`	Trigger status publish

8.7 `interactive_reward_system`

Purpose: Button-based game that dispenses a treat when the pet presses the correct button.

Telemetry payload

```
{
  "serial_number": "REWARD-001",
  "device_type": "interactive_reward_system",
```

```
  "data": {
    "pressed_button":     2,
    "winning_button":     2,
    "reward_dispensed":   true,
    "daily_reward_count": 3,
    "cooldown_active":    false,
    "button_count":       4
  }
}
```

Field	Type	Notes
`pressed_button`	integer	Which button was pressed (1-indexed)
`winning_button`	integer	The current correct button
`reward_dispensed`	boolean	Whether a reward was dispensed this event
`daily_reward_count`	integer	Total rewards given today
`cooldown_active`	boolean	Whether the cooldown timer is running
`button_count`	integer	Number of buttons on the device

Alert conditions

- `daily_reward_count >= max_rewards_per_day` (from `reward_system_configurations`, default 10) → `reward_limit_reached` INFO alert

Backend event table: `reward_events`

Commands

Command	Params	Description
`set_winning_button`	`{"button": 3}`	Set the correct button (1–5)
`dispense_reward`	`{}`	Manually dispense a reward
`set_cooldown`	`{"minutes": 5}`	Set cooldown between attempts
`set_max_rewards_per_day`	`{"max": 10}`	Set daily reward limit
`enable_game`	`{}`	Start the game
`disable_game`	`{}`	Stop the game
`get_status`	`{}`	Trigger status publish

8.8 `automatic_ball_launcher`

Purpose: Launches a ball for the pet to fetch; supports manual, scheduled, and button-triggered launches.

Telemetry payload

```
{
  "serial_number": "BALL-001",
  "device_type": "automatic_ball_launcher",
  "data": {
```

```
    "launch_source":            "app",
    "trajectory_number":        2,
    "ball_count_after_launch":  5,
    "success":                  true,
    "empty_container":          false,
    "error_message":            null
  }
}
```

Field	Type	Notes
`launch_source`	string	`"app"`, `"button"`, or `"scheduled"`
`trajectory_number`	integer	Which trajectory was used (1-indexed)
`ball_count_after_launch`	integer	Balls remaining in the container
`success`	boolean	Whether launch succeeded
`empty_container`	boolean	`true` if container is empty
`error_message`	string or null	Error detail if `success = false`

Alert conditions

- `empty_container == true` or `ball_count_after_launch <= 0` → `ball_container_empty` WARNING alert
- `success == false` → `launch_failed` WARNING alert

Backend event table: `ball_launcher_events`

Commands

Command	Params	Description
`launch_ball`	`{"trajectory_number": 2}`	Launch a ball (trajectory optional)
`set_launch_mode`	`{"mode": "manual"}`	Set mode: `manual`, `scheduled`, `both`
`set_trajectory`	`{"number": 1}`	Set default trajectory
`sync_launch_schedule`	`{}`	Push schedule to device
`get_status`	`{}`	Trigger status publish

---

9. Testing Checklist

Use this checklist when integrating a new device.

Step 1 — Backend

- [ ] Run `python app.py` (or `run_app.bat`)
- [ ] Console shows `MQTT connected to broker.emqx.io:1883`
- [ ] Open `http://127.0.0.1:5000` and log in

Step 2 — Register device

[ ] Go to **Devices** → **Register a Device**
[ ] Enter serial number (e.g. `ENV-001`) — note it exactly
[ ] Select correct device type
[ ] Assign to a pet if the type requires it (automatic_feeder does)
[ ] Click **Register Device** — confirm success message

Step 3 — Flash ESP32

[ ] Set the serial number in the sketch to match exactly (e.g. `#define SERIAL "ENV-001"`)
[ ] Set correct WiFi SSID and password
[ ] Set `mqttClient.setBufferSize(512)`
[ ] Upload sketch via Arduino IDE
[ ] Open Serial Monitor — confirm WiFi connected, MQTT connected

Step 4 — Verify device online

[ ] Wait up to 10 s after first status publish
[ ] Reload **Dashboard** — device appears with green `online` badge
[ ] Check **Telemetry** page — rows appear

Step 5 — Test telemetry

[ ] Optional: use **MQTTX** (desktop app) to publish a test payload manually:
• Connect to `broker.emqx.io:1883`
• Publish to `smartpethome/devices/ENV-001/telemetry`
• See row appear in Telemetry page within 5 s

Step 6 — Test commands

[ ] Go to **Devices** → **[device name]** (command center)
[ ] Click a command button (e.g. "Refresh Status")
[ ] Wait for ACK indicator ([OK] or [ERR]) in the Command Acknowledgements panel
[ ] Confirm ESP32 Serial Monitor shows the command was received and executed

Step 7 — Test alerts

[ ] Trigger an alert condition (e.g. turn feeder potentiometer below threshold)
[ ] Wait for alert to appear in **Alerts** page
[ ] Confirm Telegram message is received (if chat_id is configured)

---

10. Troubleshooting

Device not appearing online

Symptom	Likely cause	Fix
Device stays `offline` after ESP32 starts	Serial number mismatch	Check sketch `SERIAL` constant matches app registration exactly (case-sensitive)

Symptom	Likely cause	Fix
Device stays `offline`	ESP32 not connecting to MQTT	Check Serial Monitor for MQTT connection errors; verify WiFi credentials
Device stays `offline`	Broker ban from rapid reconnects	Add delay before reconnect: `delay(3000)` before retry loop
Device stays `offline`	Payload too large, silently dropped	Add `mqttClient.setBufferSize(512)`

Commands not received by ESP32

Symptom	Likely cause	Fix
App shows "pending" forever	ESP32 not subscribed to command topic	Confirm `mqttClient.subscribe(TOPIC_COMMAND, 1)` is called on connect
App shows "pending" forever	Wrong command topic in sketch	Verify topic exactly: `smartpethome/devices/{serial}/command`
ACK never arrives	ACK topic wrong in sketch	Verify topic exactly: `smartpethome/devices/{serial}/ack`
ACK never arrives	`command_id` not echoed	ACK payload must include `"command_id"` matching the received command

Telemetry not appearing in app

Symptom	Likely cause	Fix
Telemetry page shows nothing	Serial number unregistered	Register device in the app first
Telemetry page shows nothing	Wrong payload structure	Backend expects `data` nested object: `{"serial_number": "...", "data": {...}}`
Telemetry page shows nothing	Wrong device_type slug	Check `device_type` field in payload matches one of the 8 supported slugs

Alerts not triggering

Symptom	Likely cause	Fix
No alert despite low food	Reading above threshold	Backend threshold: reading < 1800 = critical, 1800–3000 = warning. Potentiometer must actually go below these values
No alert	Alert already unresolved	Backend deduplicates: if an unresolved alert of the same type exists, a new one is not created
Alert created but no Telegram	`telegram_chat_id` not set	Go to **Telegram** page in the app and save your Chat ID
Alert created but no Telegram	Bot token missing	Set `TELEGRAM_BOT_TOKEN` in `.env`

Symptom	Likely cause	Fix
Alert created but no Telegram	Chat ID is phone number, not numeric ID	Use `/start` on `@smartpettec_alerts_bot` to get your numeric Chat ID

Duplicate serial number error on registration

The serial number is globally unique across all users on the same Supabase instance. If you see "This serial number is already registered", either:

- You already registered this device — find it in your Devices list
- Another user registered the same serial — choose a different, unique serial number

MQTT "Banned" error in console

```
MQTT connection failed with code Banned
```

This happens when too many connections used the same client ID. The backend auto-appends a random suffix to avoid this. If it still occurs, wait 60 s for the broker to expire the old connection, then restart the app.

End of Device Integration Guide